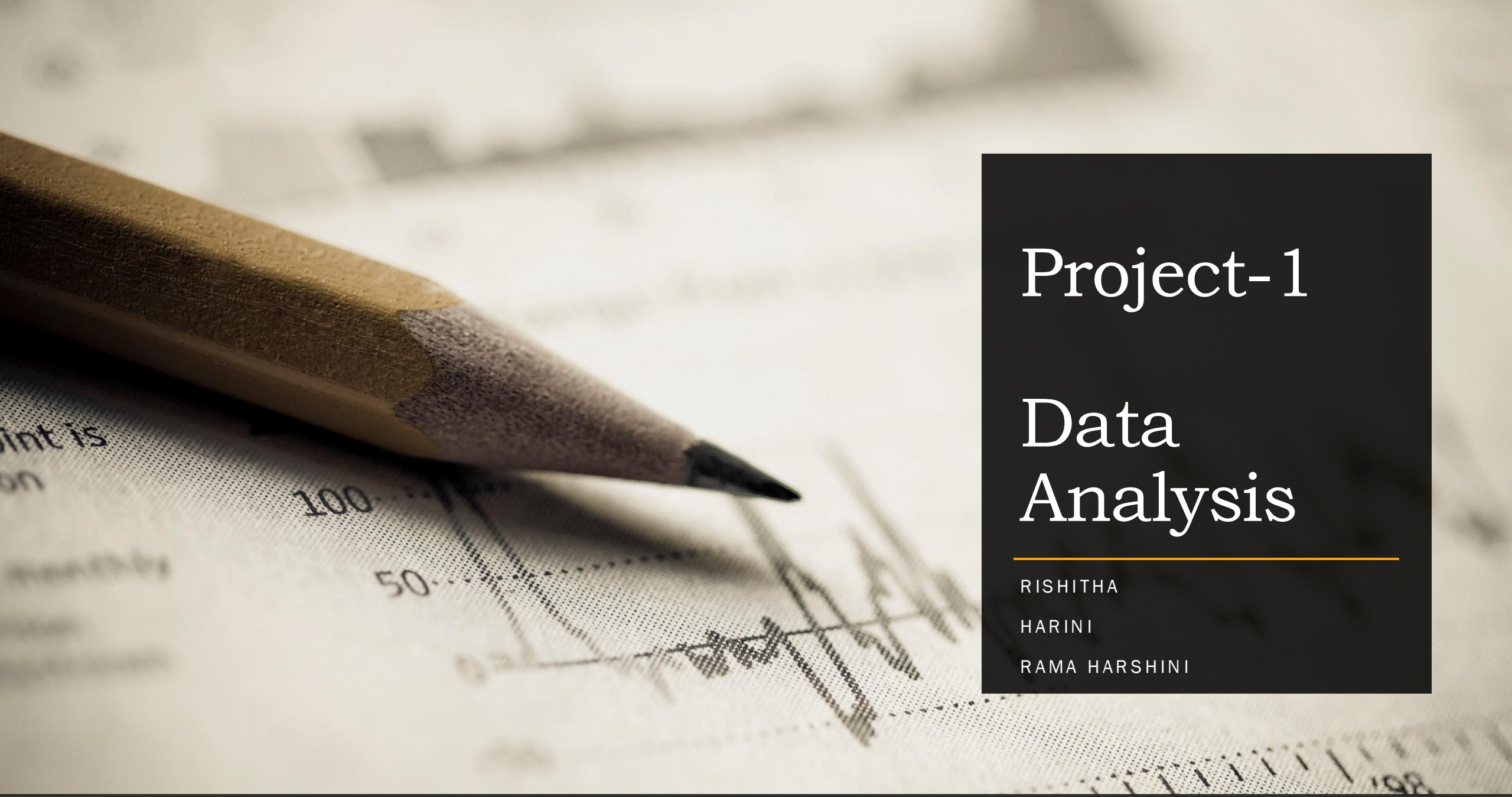


Project-1

Data Analysis

RISHITHA

HARINI

RAMA HARSHINI

We have used a real-world dataset for the project from the Kaggle.
Here we are attaching the link for the dataset we have used for this project:
https://www.kaggle.com/datasets/jeleeladekunlefijabi/ship-fuel-consumption-and-co2-emissions-analysis/data

In this dataset, there are 10 columns.

1. ship_id
2. ship_type
3. route_id
4. month
5. distance
6. fuel_type
7. fuel_consumption
8. CO2_emissions
9. weather_conditions
10. engine_efficiency

Ship_id is a unique identifier for each ship. So we don't need to do any data analysis on that column. Ship_type, route_id, month, fuel_type, weather_conditions all columns contain categorical values (a kind of discrete variable). Distance, fuel_consumption, CO2_emissions, engine_efficiency all columns contain numerical values (a kind of continuous variable).We used python for the project. You can find the link of .ipynb file in which we've done coding.
https://colab.research.google.com/drive/1W-4gsswgvjijDhmXjd6-W8wS8ToAxUbF#scrollTo=i4JdKhY8Vi4N

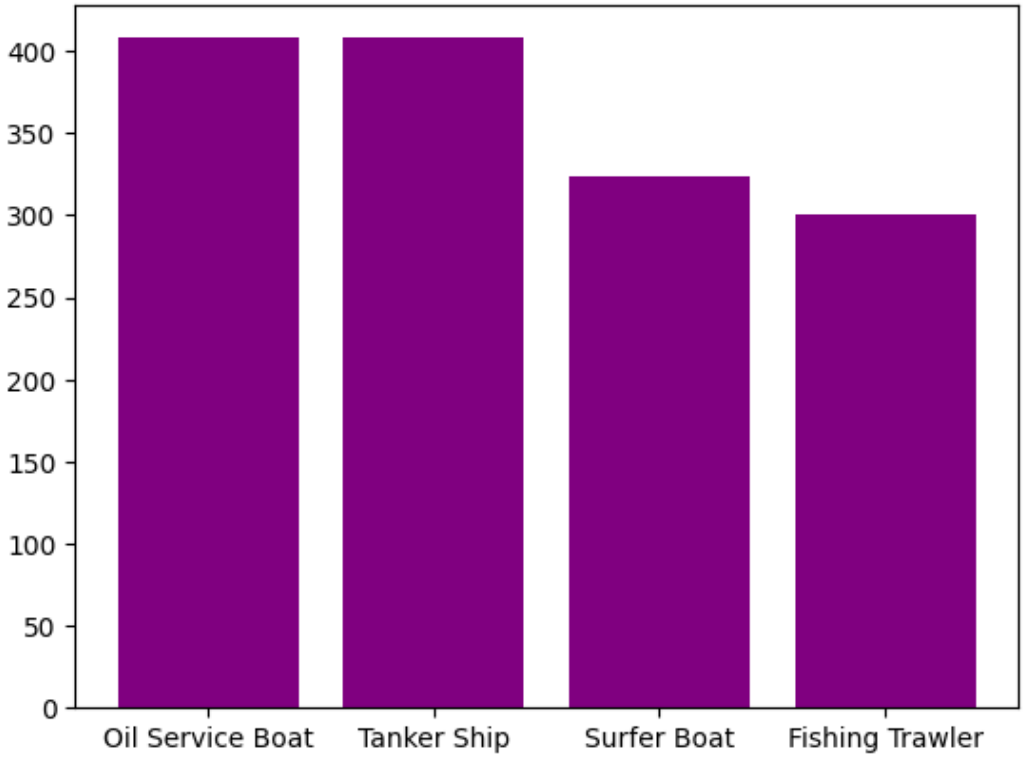
PART-1

1. Bar plots
2. Ogive curves
3. Histograms
4. Pie charts
5. Boxplots

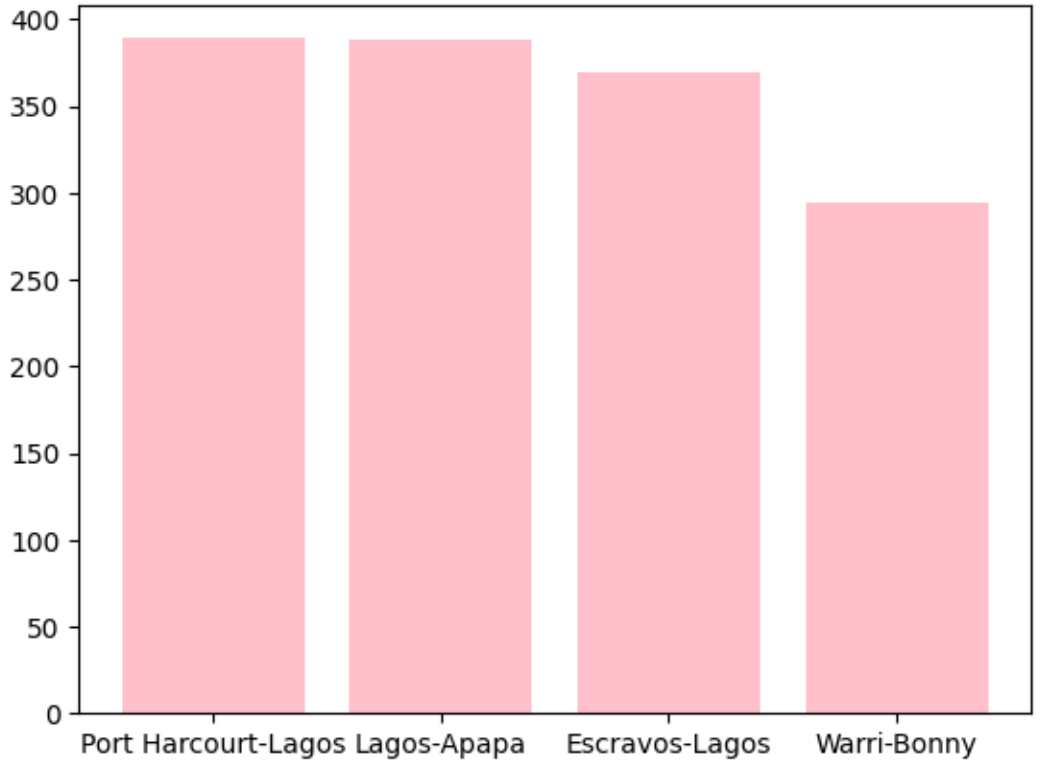
Bar diagrams

Codes used to generate the bar plots:

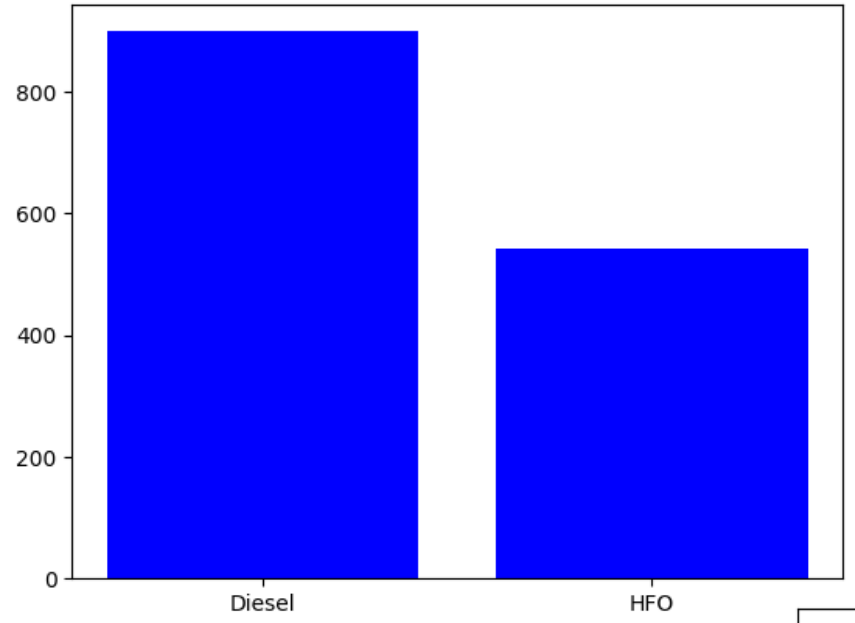
```
#Barplot over ship type
plt.bar(data['ship_type'].value_counts().index,data['ship_type'].value_counts().values,color='purple’)
plt.show()


#Bar plot over route_id
plt.bar(data['route_id'].value_counts().index,data['route_id'].value_counts().values)
plt.show()


#Bar plot over month
plt.bar(data['month'].value_counts().index,data['month'].value_counts().values)
plt.show()


#Bar plot over fuel
plt.bar(data['fuel_type'].value_counts().index,data['fuel_type'].value_counts().values)
plt.show()


#Barplot over weather_conditions
plt.bar(data['weather_conditions'].value_counts().index,data['weather_conditions'].value_counts(
).values)
plt.show()
```



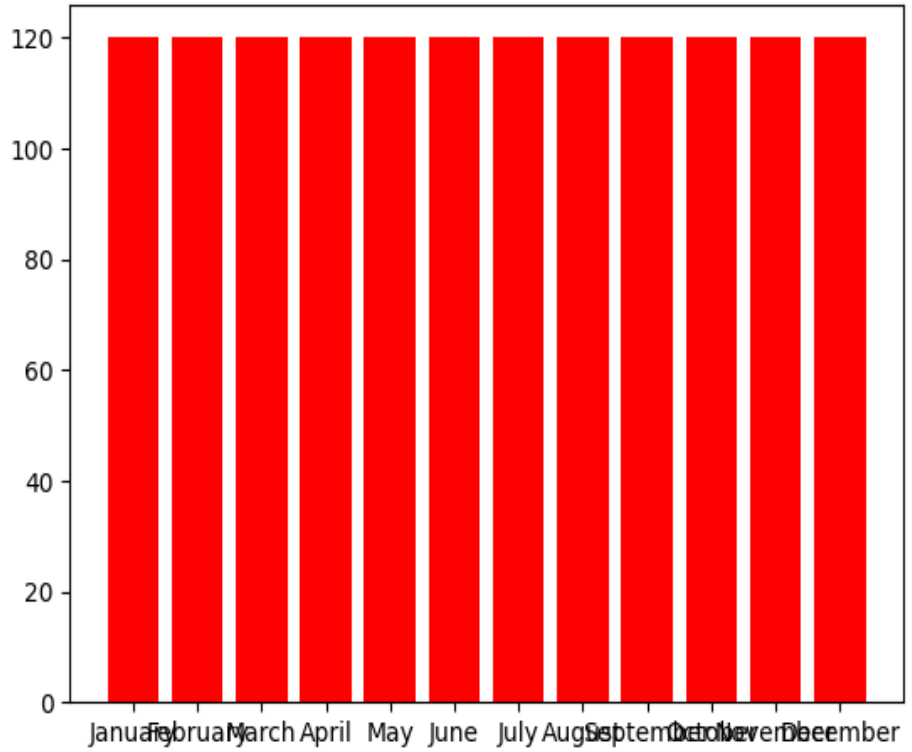
Barplot over the column ship_type



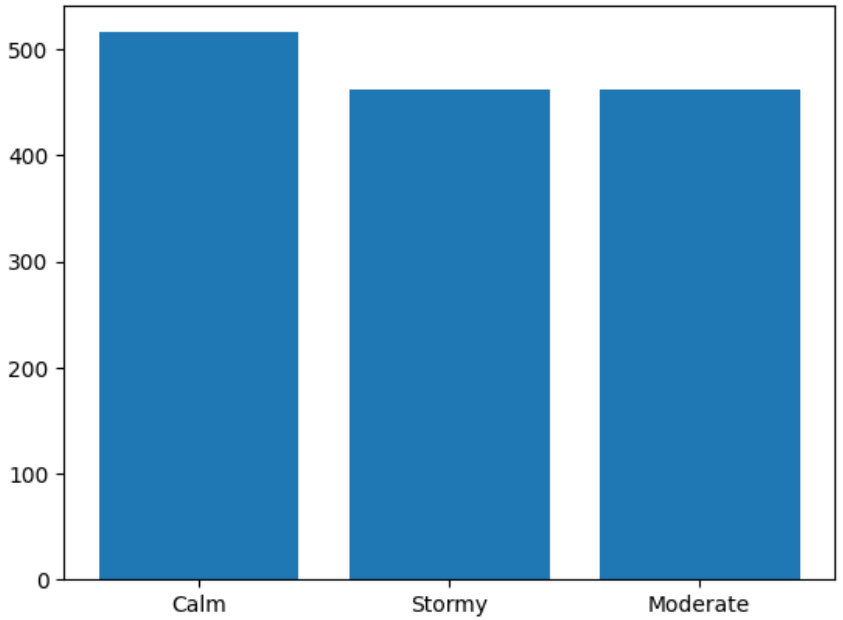
Barplot over the column route_id



Barplot over the column “fuel_type”



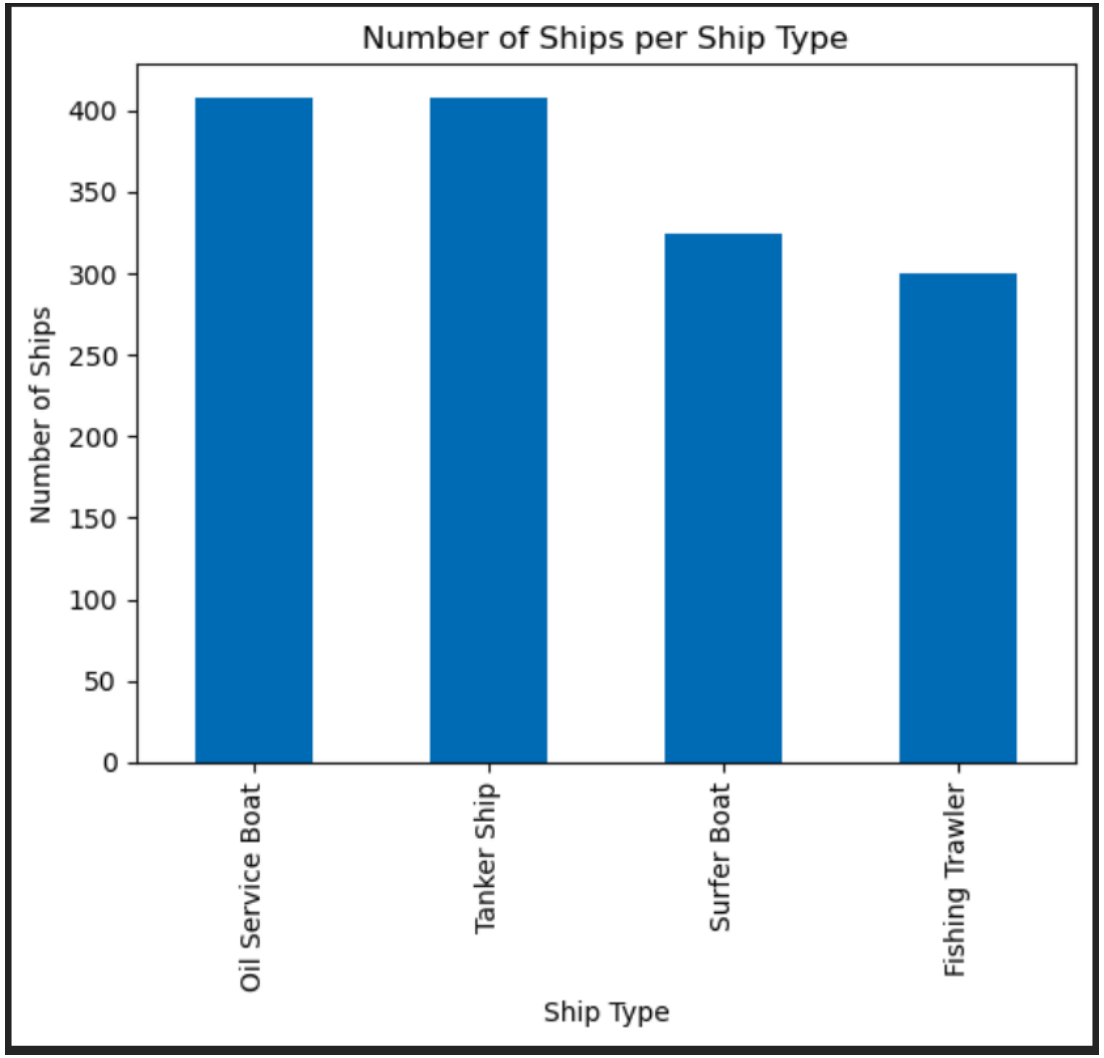
Barplot over the column “month”



Barplot over the column “weather_conditions”

Code to display the number of ships of different types:

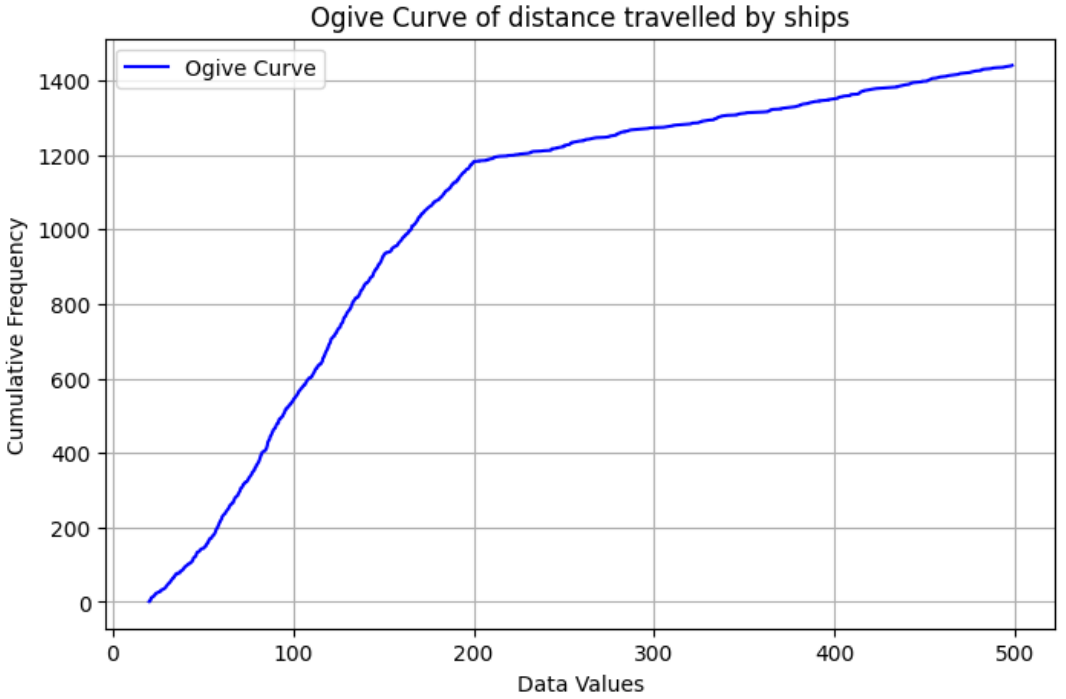
```
data['ship_type'].value_counts().plot(kind='bar')
plt.xlabel('Ship Type')
plt.ylabel('Number of Ships')
plt.title('Number of Ships per Ship Type')
plt.show()
```



Ogive Curves

Code used:

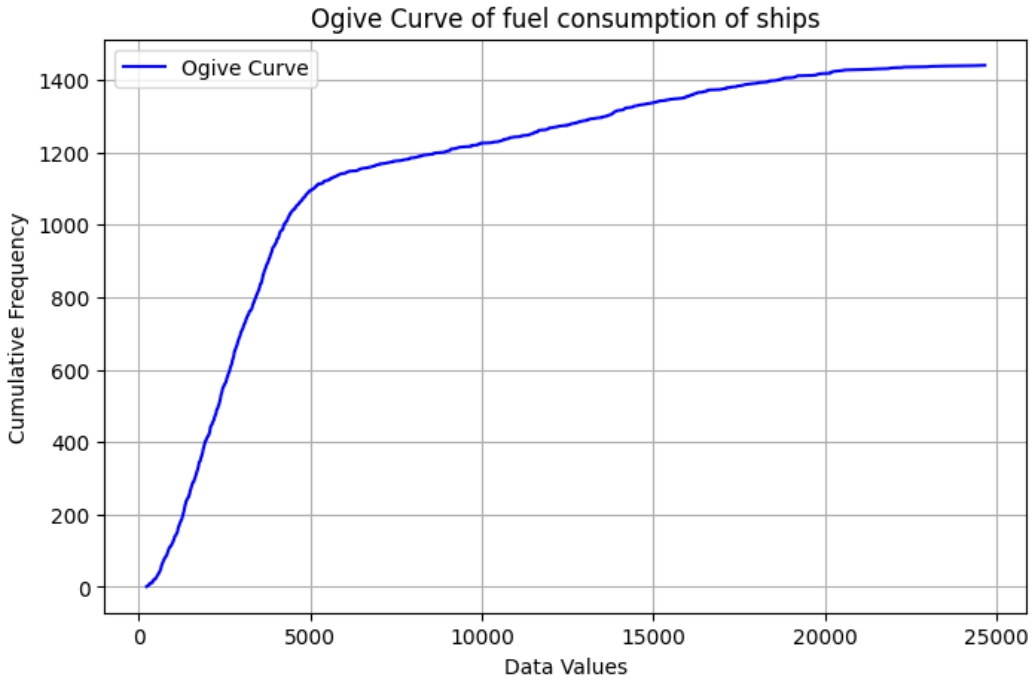
```
#Ogive curve for distace column
distances = data['distance'].sort_values().values

values, frequency = np.unique(distances, return_counts = True)
cf_distances = np.cumsum(frequency)

plt.figure(figsize=(8, 5))
plt.plot(values, cf_distances, linestyle="-", color="b", label="Ogive Curve")
plt.xlabel("Data Values")
plt.ylabel("Cumulative Frequency")
plt.title("Ogive Curve of distance travelled by ships")
plt.grid(True)
plt.legend()
plt.show()
```



From the Ogive curve plotted over the distance column , we can observe that gradient after 200 is less increasing than before which means less percentage of data falls in that region

Code used:

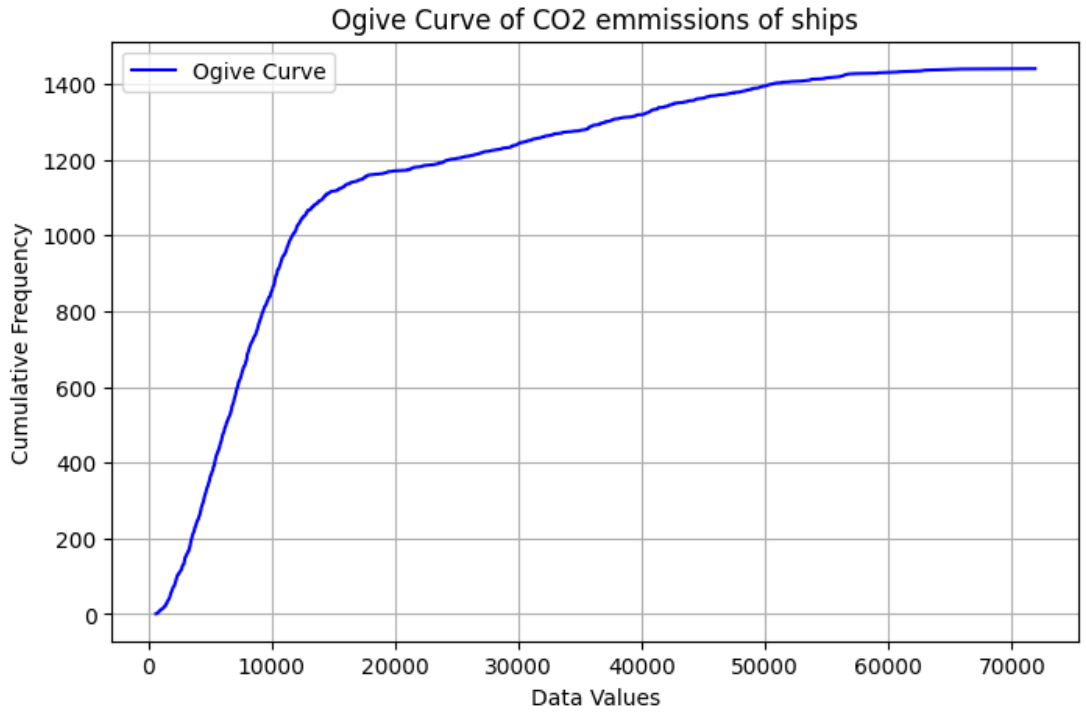
```
#Ogive curve for fuel_consumption column
fuel_con = data['fuel_consumption'].sort_values().values
values, frequency = np.unique(fuel_con, return_counts = True)
cf_fuel_con = np.cumsum(frequency)
plt.figure(figsize=(8, 5))
plt.plot(values, cf_fuel_con, linestyle="-", color="b", label="Ogive Curve")
plt.xlabel("Data Values")
plt.ylabel("Cumulative Frequency")
plt.title("Ogive Curve of fuel consumption of ships")
plt.grid(True)
plt.legend()
plt.show()
```



From the Ogive curve plotted over the fuel consumption column , we can observe that gradient after 5000 is less increasing than before which means more ships consume less than 5000 units of fuel

Code used:

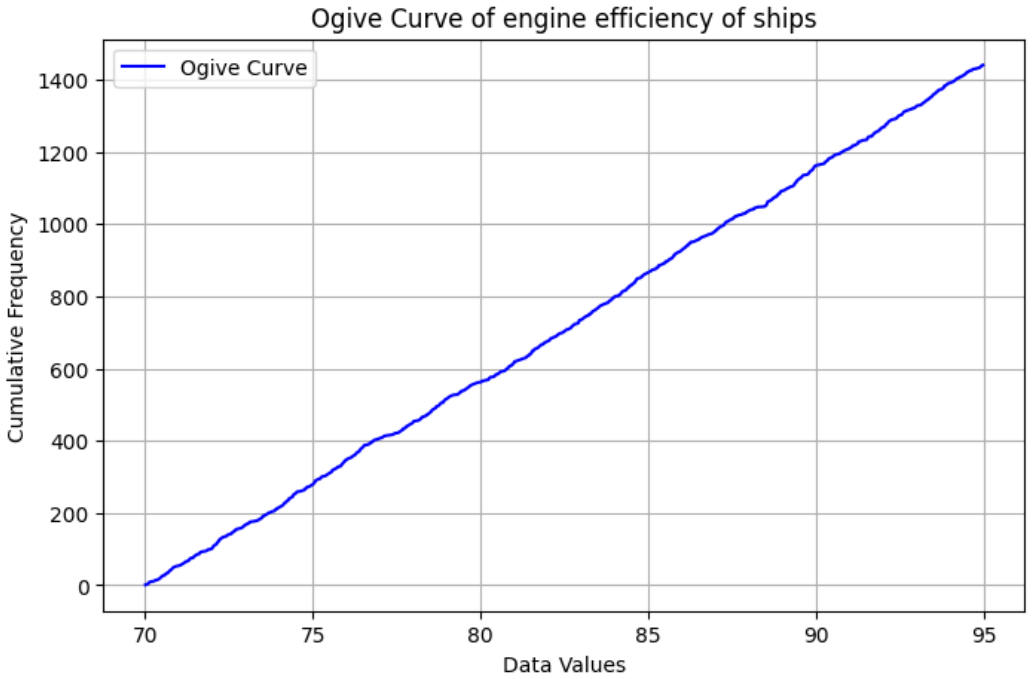
```
#Ogive curve for CO2_emissions column
CO2_em = data['CO2_emissions'].sort_values().values

values, frequency = np.unique(CO2_em, return_counts = True)
cf_CO2_em = np.cumsum(frequency)

plt.figure(figsize=(8, 5))
plt.plot(values, cf_CO2_em, linestyle="-", color="b", label="Ogive Curve")
plt.xlabel("Data Values")
plt.ylabel("Cumulative Frequency")
plt.title("Ogive Curve of CO2 emmissions of ships")
plt.grid(True)
plt.legend()
plt.show()
```



From the Ogive curve plotted over the fuel consumption column , we can observe that gradient after 15000 is less increasing than before which means more ships emit less than 15000 units of C02

Code used:

```
#Ogive curve for engine efficiency column
eng_eff = data['engine_efficiency'].sort_values().values

values, frequency = np.unique(eng_eff, return_counts = True)
cf_eng_eff = np.cumsum(frequency)

plt.figure(figsize=(8, 5))
plt.plot(values, cf_eng_eff, linestyle="-", color="b", label="Ogive Curve")
plt.xlabel("Data Values")
plt.ylabel("Cumulative Frequency")
plt.title("Ogive Curve of engine efficiency of ships")
plt.grid(True)
plt.legend()
plt.show()
```

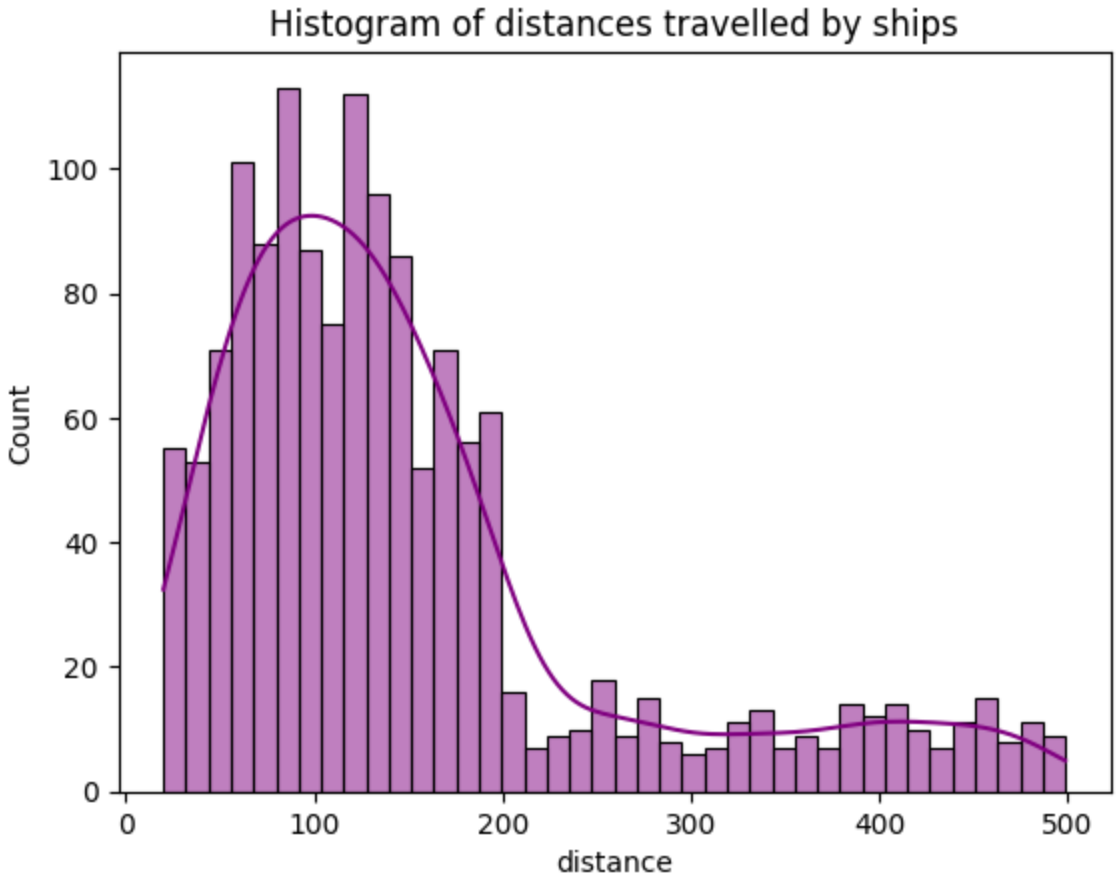


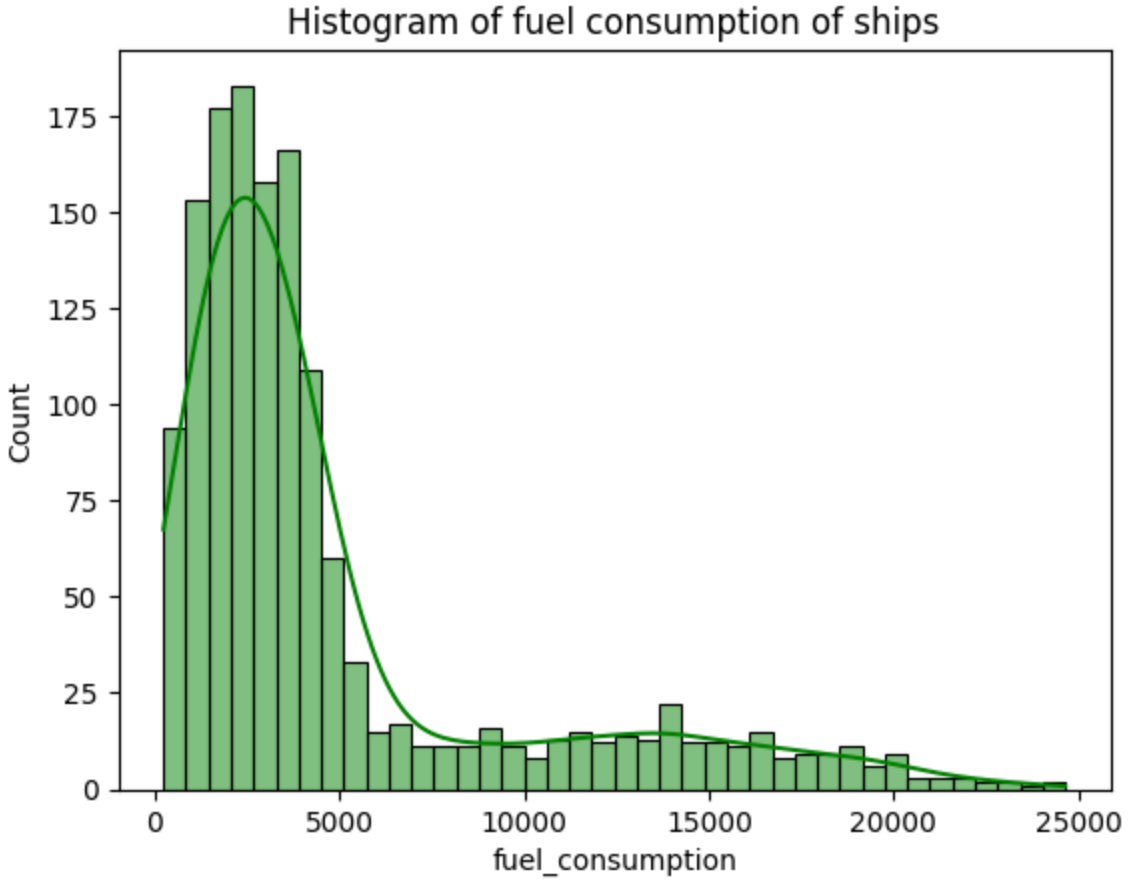
The Ogive curve of the engine efficiency column is almost a straight line which indicates the data is very well uniformly distributed across the range

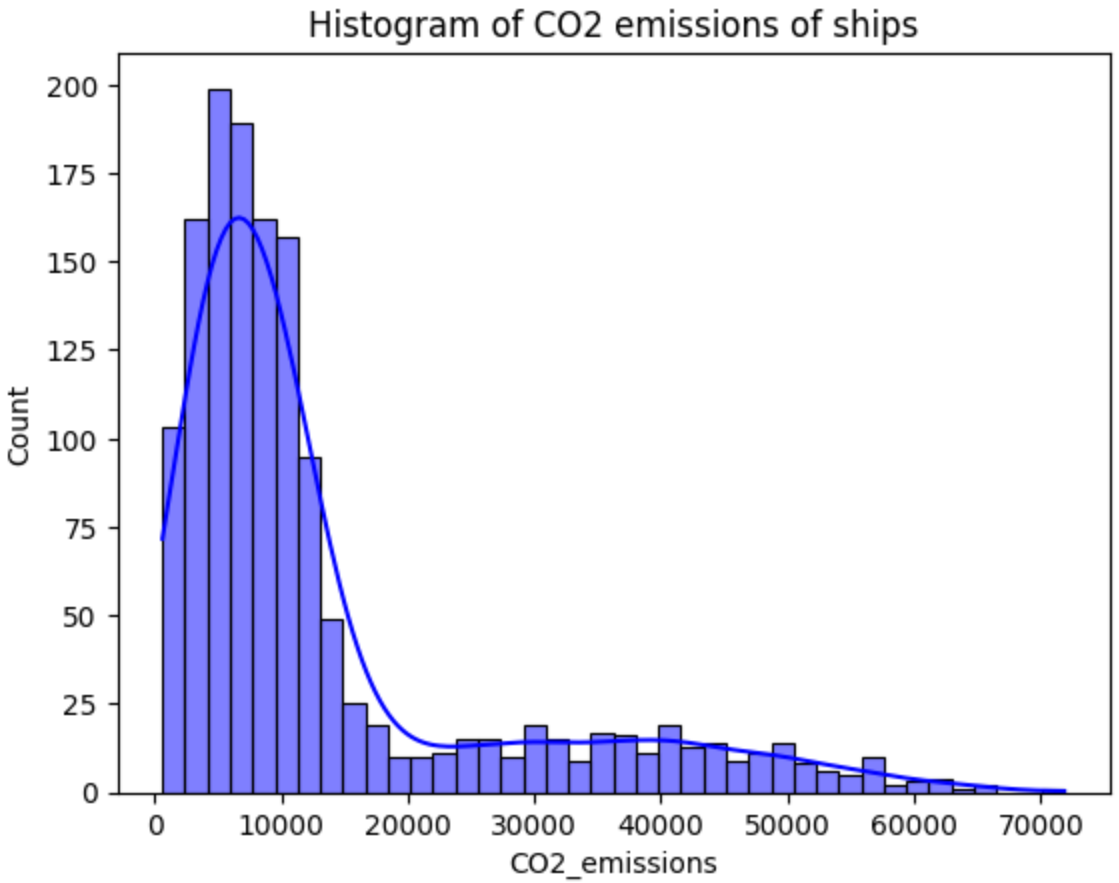
Histograms

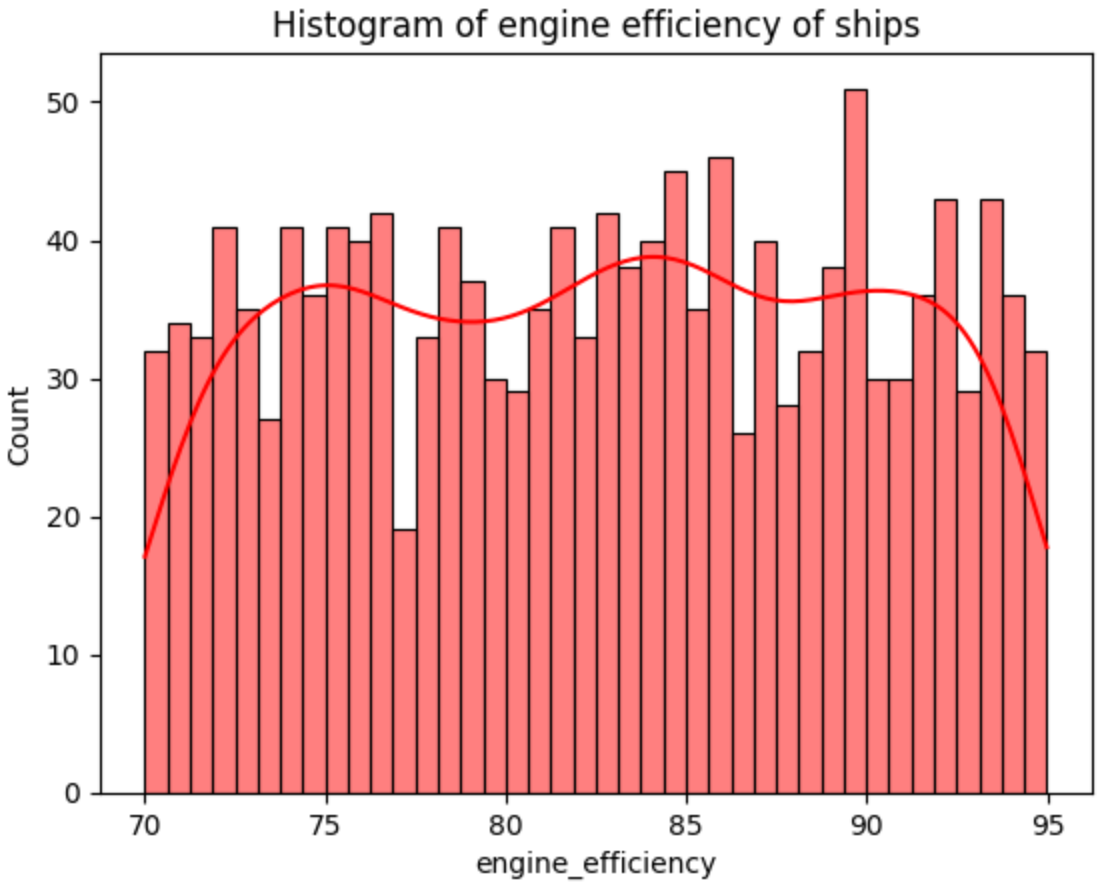
Code used to generate the histograms:

```
#Histogram for distance column
plt.title("Histogram of distances travelled by ships")
sns.histplot(data['distance'], bins=40, edgecolor='black', kde=True,color='purple')
plt.show()


#Histogram over fuel_consumption column
plt.title("Histogram of fuel consumption of ships")
sns.histplot(data['fuel_consumption'], bins=40, edgecolor='black', kde=True,color='green')
plt.show()


#Histogram over CO2 emissions column
plt.title("Histogram of CO2 emissions of ships")
sns.histplot(data['CO2_emissions'], bins=40, edgecolor='black', kde=True,color='blue')
plt.show()


#Histogram over enginne efficiency column
plt.title("Histogram of engine efficiency of ships")
sns.histplot(data['engine_efficiency'], bins=40, edgecolor='black', kde=True,color='red')
plt.show()
```


Histogram of distances travelled by ships
Count
distance
0
20
40
60
80
100
0
100
200
300
400
500


Histogram of fuel consumption of ships
Count
fuel_consumption
0
25
50
75
100
125
150
175
0
5000
10000
15000
20000
25000


Histogram of CO2 emissions of ships
Count
CO2_emissions
0
25
50
75
100
125
150
175
200
0
10000
20000
30000
40000
50000
60000
70000

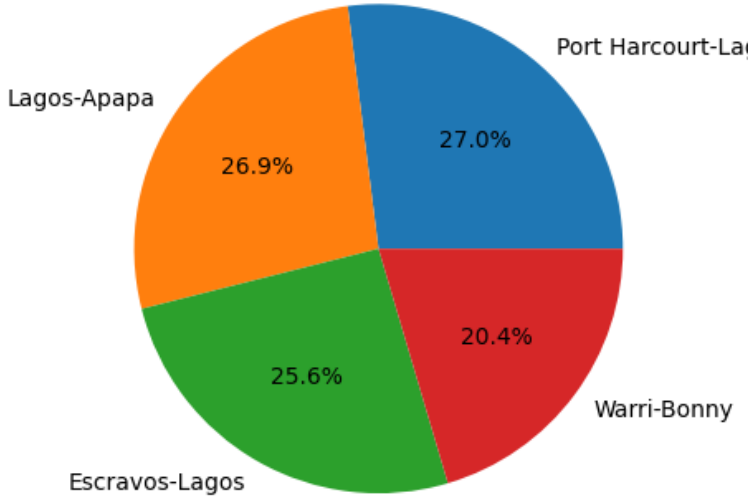

Histogram of engine efficiency of ships
Count
engine_efficiency
0
10
20
30
40
50
70
75
80
85
90
95

Pie Charts

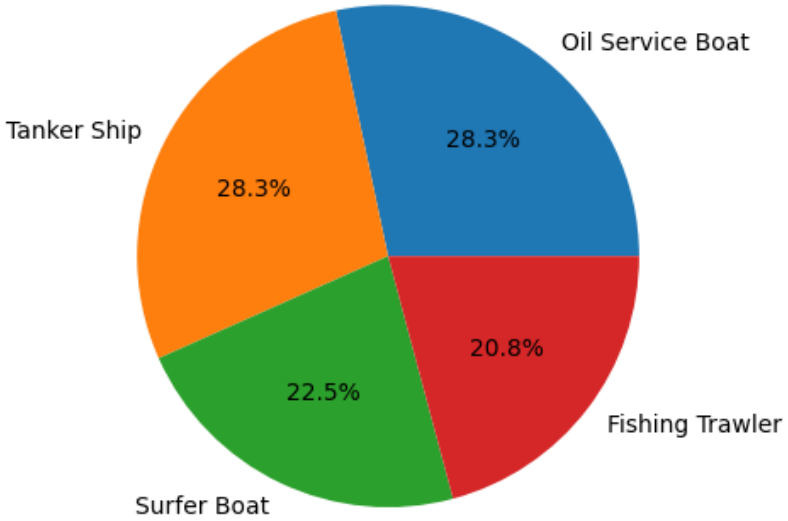
Codes used to generate the piecharts:

```
#piechart over ship_type column
plt.pie(data['ship_type'].value_counts(),labels=data['ship_type'].value_counts().index,autopct='%1.1f%%')
plt.title('Piechart over the column "Ship type"')
plt.show()

#piechart over route_id column
plt.pie(data['route_id'].value_counts(),labels=data['route_id'].value_counts().index,autopct='%1.1f%%')
plt.title('Piechart of the column "route_id"')
plt.show()

#piechart over month column
plt.pie(data['month'].value_counts(),labels=data['month'].value_counts().index,autopct='%1.1f%%')
plt.title('Piechart of the column "month"')
plt.show()

#piechart over fuel_type column
plt.pie(data['fuel_type'].value_counts(),labels=data['fuel_type'].value_counts().index,autopct='%1.1f%%')
plt.title('Piechart of the column "fuel_type"')
plt.show()

#piechart over weather_conditions column
plt.pie(data['weather_conditions'].value_counts(),labels=data['weather_conditions'].value_counts().index,autopct='%
1.1f%%')
plt.title('Piechart of the column "weather_conditions"')
plt.show()
```

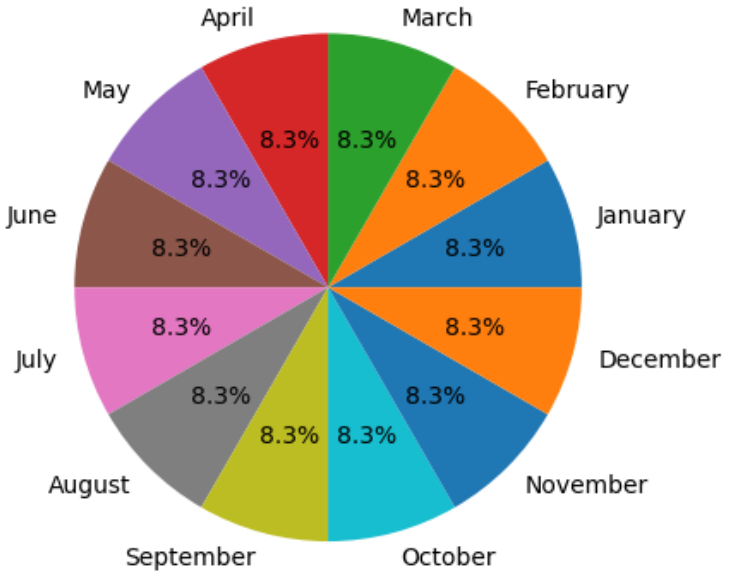
Piechart of the column "route_id"



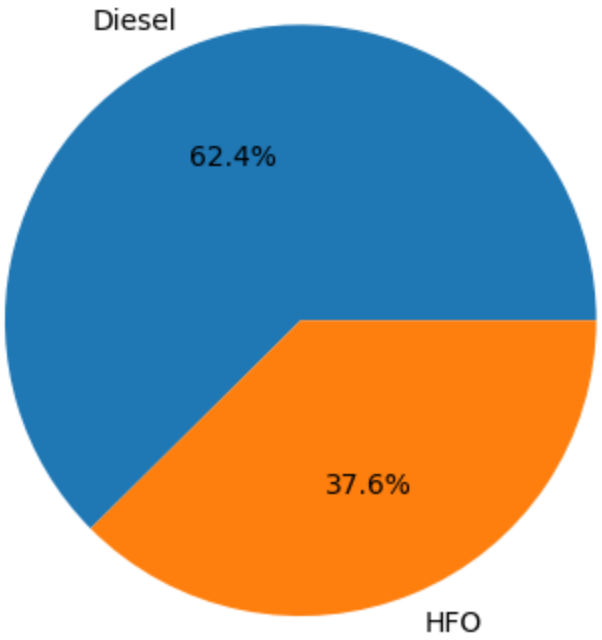
Piechart over the column "Ship type"



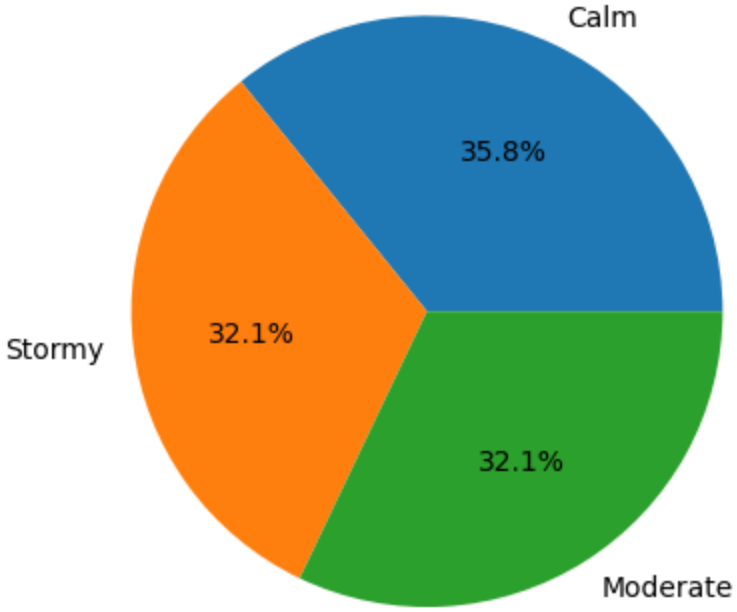
Piechart of the column "month"



Piechart of the column "fuel_type"



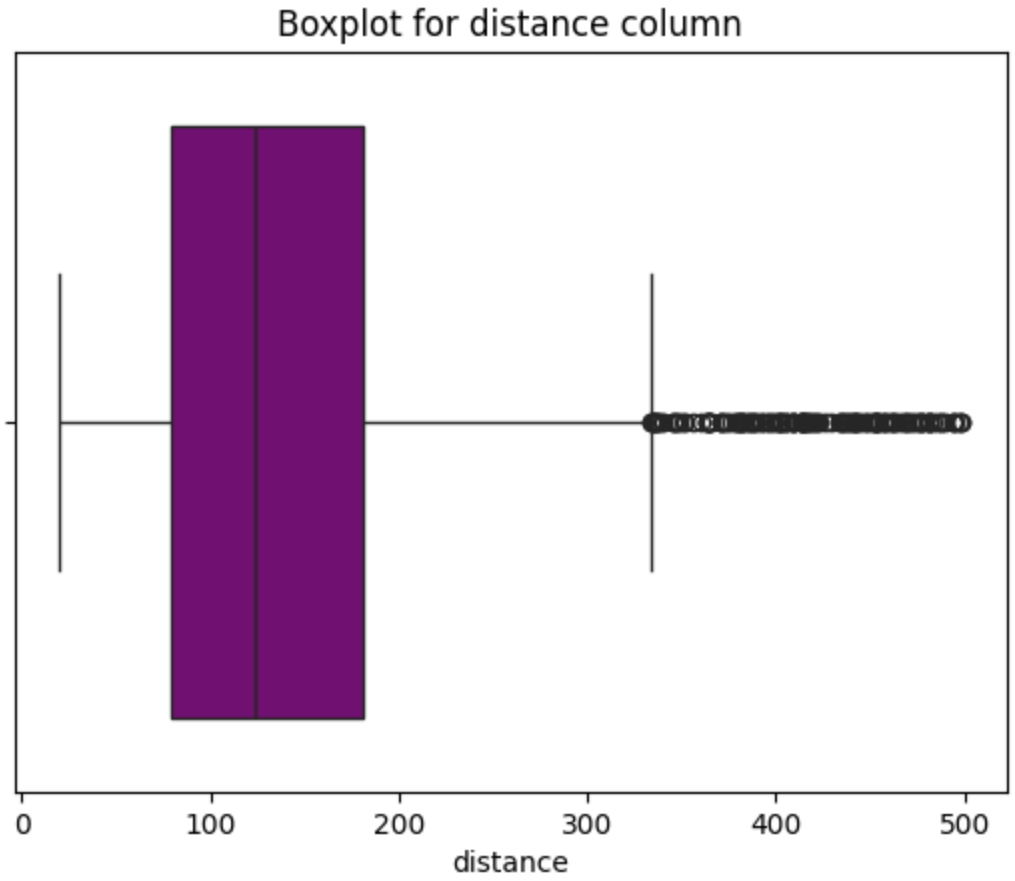
Piechart of the column "weather_conditions"

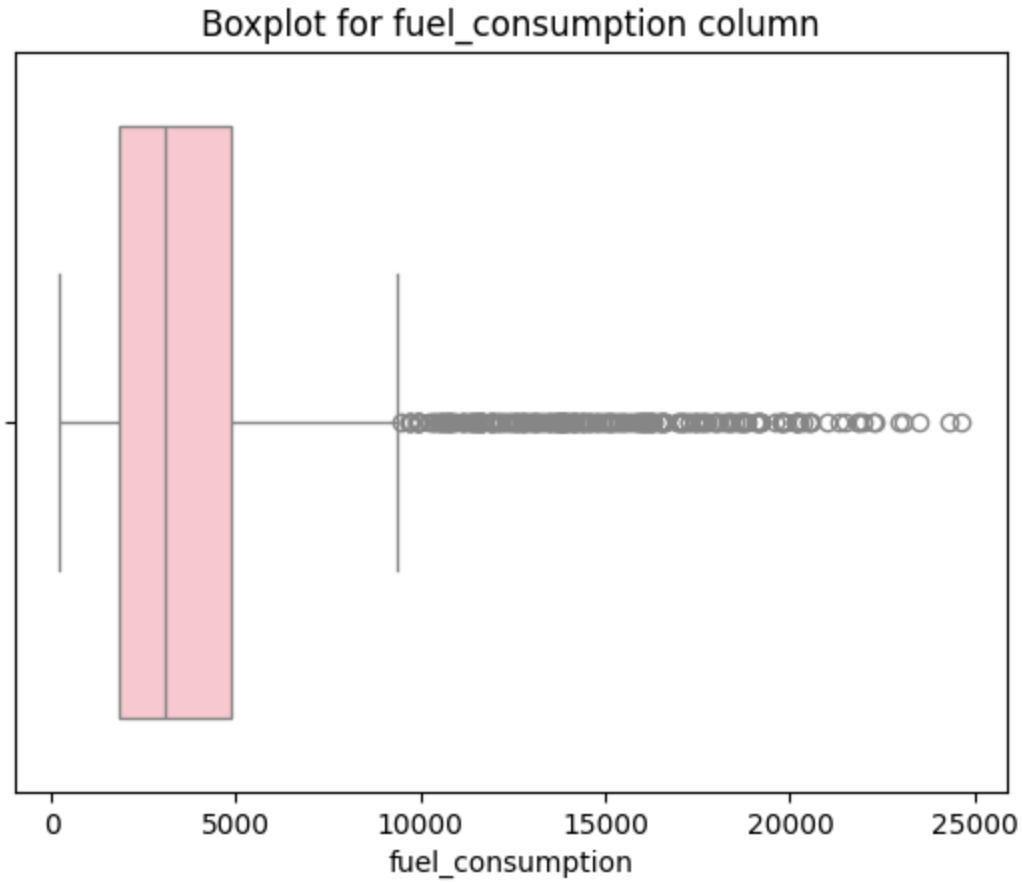


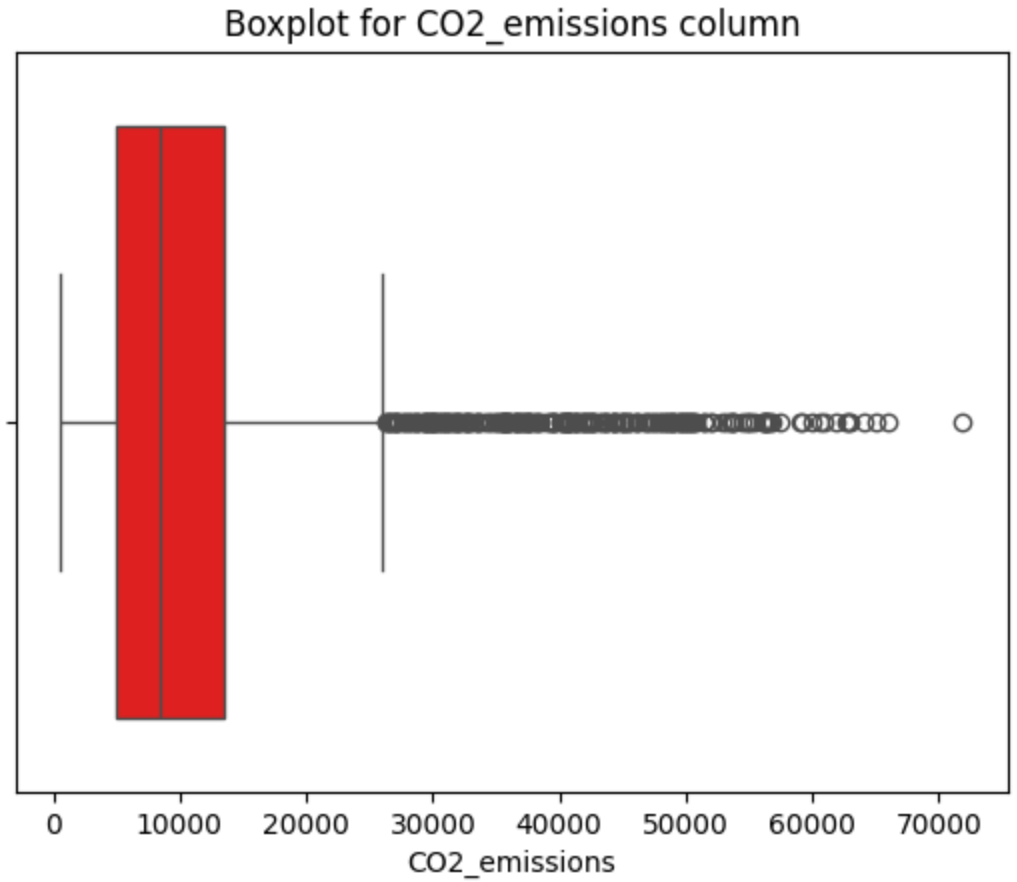
Box Plots

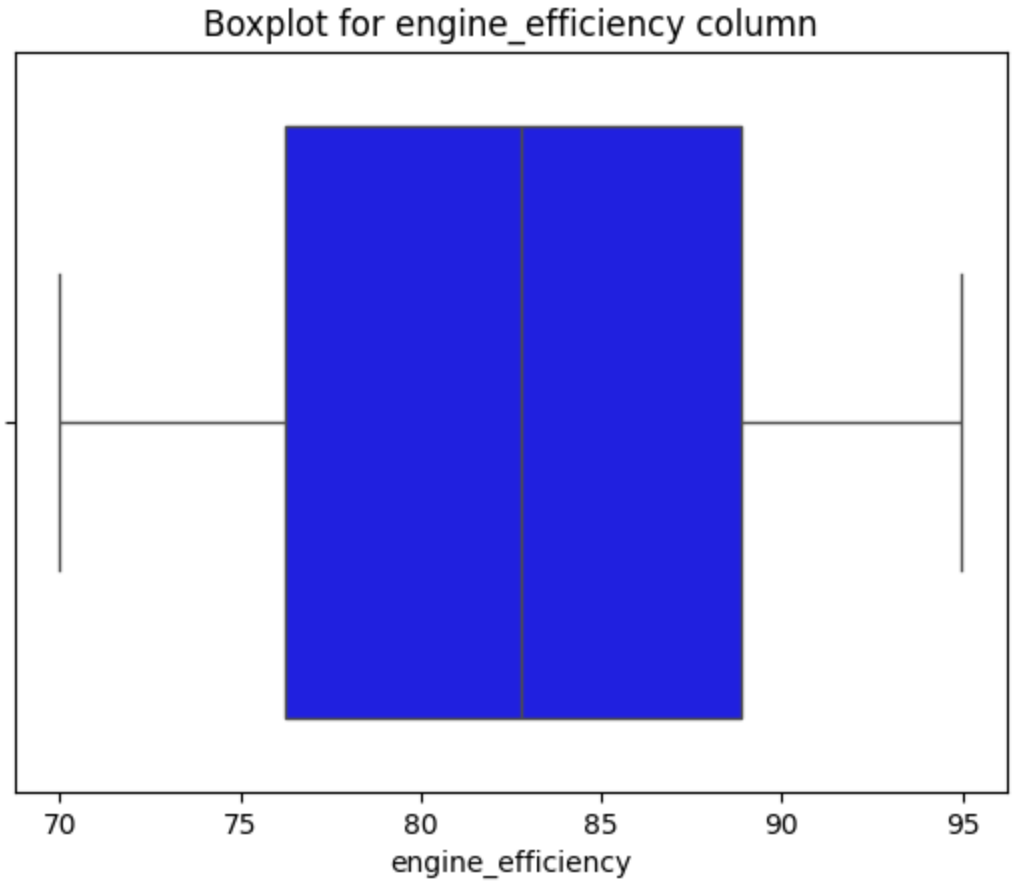
Codes used to generate the Boxplots:

```
#Boxplot for distance column
sns.boxplot(x=data['distance'],color='purple')
plt.title("Boxplot for distance column")
plt.show()


#Boxplot for fuel_consmption column
sns.boxplot(x=data['fuel_consumption'],color='pink')
plt.title("Boxplot for fuel_consumption column")
plt.show()


#Boxplot for CO2_emissions column
sns.boxplot(x=data['CO2_emissions'],color='red')
plt.title("Boxplot for CO2_emissions column")
plt.show()


#Boxplot over engine_efficiency column
sns.boxplot(x=data['engine_efficiency'],color='blue')
plt.title("Boxplot for engine_efficiency column")
plt.show()
```


Boxplot for distance column
0
100
200
300
400
500
distance


Boxplot for fuel_consumption column
0
5000
10000
15000
20000
25000
fuel_consumption


Boxplot for CO2_emissions column
0
10000
20000
30000
40000
50000
60000
70000
CO2_emissions


Boxplot for engine_efficiency column
70
75
80
85
90
95
engine_efficiency

Observations from boxplots:

Boxplot for Distance Column:

The data is right-skewed, with a significant number of outliers on the higher end, indicating a few instances of extremely large distances.
The interquartile range (IQR) is concentrated in the lower values, suggesting that most distances are relatively small.

Boxplot for Fuel Consumption Column:

The distribution shows a strong right skew, with a large number of outliers at the end.
The median fuel consumption is relatively low

Boxplot for CO2 Emissions Column:

The data is heavily right-skewed, with many outliers at the right end
Most CO2 emissions values fall within a compact range

Boxplot for Engine Efficiency Column:

The data appears to be more symmetrically distributed compared to other columns, with a balanced median within the IQR
There are no significant outliers

Part-2

Descriptive summary of numerical columns:

1. Measures of central tendency:
    1. Mean
    2. Median
    3. Mode
2. Measures of dispersion
3. Quartiles

Measures of central tendencies of distance column:

```
import statistics as stats
avg_dist = data['distance'].mean()
print(f'Mean of distance travelled by ships = {avg_dist}')
median_dist = data['distance'].median()
print(f'Median of distance travelled by ships = {median_dist}')
mode_dist = stats.mode(data['distance'])
print(f'Mode of distance travelled by ships = {mode_dist}')
```

Output:

```
Mean of distance travelled by ships= 151.75

Median of distance travelled by ships= 123.465

Mode of distance travelled by ships = 21.11
```

Measures of dispersion of distance column:

```
#Range of data in distance column
range_dist = np.ptp(data['distance'])
print(f'Range of distances travelled by ships = {range_dist}')
#Variance of data in distance column
var_dist = data['distance'].var()
print(f'Variance of distance travelled by ships = {var_dist}')
#Std deviation of data in distance column
std_dist = data['distance'].std()
print(f'Standard deviation of distance travelled by ships = {std_dist}')
```

Output:

```
Range of distances travelled by ships= 478.47

Var of distance travelled by ships= 11766.23

Std deviation of distance travelled by ships=
108.47222953325979
```

Quartiles of distance column:

```
#Quartile range for distance column:
Q1_dist = data['distance'].quantile(0.25)
Q2_dist = data['distance'].quantile(0.50)
Q3_dist = data['distance'].quantile(0.75)
IQR_dist = Q3_dist - Q1_dist
print("Quartiles of distance travelled by ships:")
print(f'Q1, Q2, Q3 = {Q1_dist}, {Q2_dist}, {Q3_dist}')
print(f'Interquartile range of distance travelled by ships = {IQR_dist}')
```

Output:

```
Quartiles of distance travelled by ships:
Q1, Q2, Q3 = 79.0025, 123.465, 180.78

Interquartile range of distance travelled by
ships = 101.7775
```

Measures of central tendencies of fuel consumption column:

```
mean_fuel_consumption = data['fuel_consumption'].mean()
print(f'Mean of fuel consumption of ships = {mean_fuel_consumption}')
median_fuel_consumption = data['fuel_consumption'].median()
print(f'Median of fuel consumption of ships =
{median_fuel_consumption}')
mode_fuel_consumption = stats.mode(data['fuel_consumption'])
print(f'Mode of fuel consumption of ships = {mode_fuel_consumption}')
```

Output:

```
Mean of fuel consumption of ships
= 4844.246534722222

Median of fuel consumption of ships = 3060.88

Mode of fuel consumption of ships = 855.01
```

Measures of dispersions of fuel consumption column:

```
#Range of data in fuel consumption column
range_fuel_consumption = np.ptp(data['fuel_consumption'])
print(f'Range of fuelconsumption of ships = {range_fuel_consumption}')
#Variance of data in fuel consumption column
var_fuel_consumption = data['fuel_consumption'].var()
print(f'Variance of fuelconsumption of ships = {var_fuel_consumption}')
#Std deviation of data in fuel consumption column
std_fuel_consumption = data['fuel_consumption'].std()
print(f'Standard deviation of fuelconsumption of ships ={std_fuel_consumption}')
```

Output:

```
Range of fuel consumption
= 24410.64

Variance of fuel consumption
= 23935116.042488102

Standard deviation of fuel consumption
= 4892.35281255227
```

Quartiles of fuel consumption column:

```
#Quartile range for fuel consumption column:
Q1_fuel_consumption = data['fuel_consumption'].quantile(0.25)
Q2_fuel_consumption = data['fuel_consumption'].quantile(0.50)
Q3_fuel_consumption = data['fuel_consumption'].quantile(0.75)
IQR_fuel_consumption = Q3_fuel_consumption- Q1_fuel_consumption
print("Quartiles of fuel_consumption of ships:")
print(f'Q1, Q2, Q3 = {Q1_fuel_consumption}, {Q2_fuel_consumption}, {Q3_fuel_consumption}')
print(f'Interquartile range of fuel_consumption of ships = {IQR_fuel_consumption}')
```

Output:

```
Quartiles of fuel consumption:

Q1, Q2, Q3, = 1837.9625, 3060.88, 4870.675

Interquartile range of fuel consumption of
ships = 3032.7125
```

Measures of central tendencies of CO2 emissions column:

```
mean_CO2_emissions = data['CO2_emissions'].mean()
print(f'Mean of CO2 emissions of ships = {mean_CO2_emissions}')
median_CO2_emissions = data['CO2_emissions'].median()
print(f'Median of CO2 emissions of ships = {median_CO2_emissions}')
Mode_CO2_emissions = stats.mode(data['CO2_emissions'])
print(f'Mode of CO2 emissions of ships = {mode_CO2_emissions}')
```

Output:

```
Mean of CO2 emissions of ships
= 13365.454881944444

Median of CO2 emissions of ships =
8423.255000000001

Mode of CO2 emissions of ships = 10625.76
```

Measures of dispersions of CO2 emissions column:

```
#Range of data in CO2 emissions column
range_CO2_emissions = np.ptp(data['CO2_emissions'])
print(f'Range of CO2 emissions  of ships = {range_CO2_emissions}')
#Variance of data in CO2 emissions column
var_CO2_emissions = data['CO2_emissions'].var()
print(f'Variance of CO2 emissions  of ships = {var_CO2_emissions}')
#Std deviation of data in CO2 emissions column
std_CO2_emissions = data['CO2_emissions'].std()
print(f'Standard deviation of CO2 emissions of ships ={std_CO2_emissions}')
```

Output:

```
Range of CO2 emissions of ships
= 71255.53000000001

Variance of CO2 emissions of ships
= 184081129.73245686

Standard deviation of CO2 emissions of ships
= 13567.650118294503
```

Quartiles of CO2 emissions column:

```
#Quartile range for CO2 emissions column:
Q1_CO2_emissions = data['CO2_emissions'].quantile(0.25)
Q2_CO2_emissions = data['CO2_emissions'].quantile(0.50)
Q3_CO2_emissions = data['CO2_emissions'].quantile(0.75)
IQR_CO2_emissions = Q3_CO2_emissions- Q1_CO2_emissions
print("Quartiles of CO2 emissions of ships:")
print(f'Q1, Q2, Q3 = {Q1_CO2_emissions}, {Q2_CO2_emissions}, {Q3_CO2_emissions}')
print(f'Interquartile range of CO2 emissions of ships = {IQR_CO2_emissions}')
```

Output:

```
Quartiles of CO2 emissions:

Q1, Q2, Q3, = 4991.485, 8423.255000000001,
13447.12

Interquartile range of CO2 emissions of ships =
8455.635000000002
```

Measures of central tendencies of engine efficiency column:

```
mean_engine_efficiency = data['engine_efficiency'].mean()
print(f'Mean of engine efficiency of ships = {mean_engine_efficiency}')
median_engine_efficiency = data['engine_efficiency'].median()
print(f'Median of engine efficiency of ships = {median_engine_efficiency}')
Mode_engine_efficiency = stats.mode(data['engine_efficiency'])
print(f'Mode of engine efficiency of ships = {mode_engine_efficiency}')
```

Output:

```
Mean of engine efficiency of ships
= 82.58292361111111

Median of engine efficiency of ships = 82.775

Mode of engine efficiency of ships = 82.94
```

Measures of dispersions of engine efficiency column:

```
#Range of data in engine efficiency column
range_engine_efficiency = np.ptp(data['engine_efficiency'])
print(f'Range of engine efficiency  of ships = {range_engine_efficiency}')
#Variance of data in engine efficiency column
var_engine_efficiency = data['engine_efficiency'].var()
print(f'Variance of engine efficiency  of ships = {var_engine_efficiency}')
#Std deviation of data in engine efficiency column
std_engine_efficiency = data['engine_efficiency'].std()
print(f'Standard deviation of engine efficiency of ships ={std_engine_efficiency}')
```

Output:

```
Range of engine efficiency of ships
= 24.97

Variance of engine efficiency of ships
= 51.24110715190915

Standard deviation of engine efficiency of
ships
= 7.158289401240295
```

Quartiles of engine efficiency column:

```
#Quartile range for engine efficiency column:
Q1_engine_efficiency = data['engine_efficiency'].quantile(0.25)
Q2_engine_efficiency = data['engine_efficiency'].quantile(0.50)
Q3_engine_efficiency = data['engine_efficiency'].quantile(0.75)
IQR_engine_efficiency = Q3_engine_efficiency- Q1_engine_efficiency
print("Quartiles of engine efficiency of ships:")
print(f'Q1, Q2, Q3 = {Q1_engine_efficiency}, {Q2_engine_efficiency},
{Q3_engine_efficiency}')
print(f'Interquartile range of engine efficiency of ships = {IQR_engine_efficiency}')
```

Output:

```
Quartiles of engine efficiency:

Q1, Q2, Q3, = 76.255, 82.775, 88.8625

Interquartile range of engine efficiency of
ships = 12.607500000000002
```

Observations of central tendencies and dispersions:

FOR THE DISTANCE COLUMN:

- As the mean is higher than the median of distances travelled the graph is right skewed.
- Since the mode is lower than the mean and median, it implies that most of the ships travel shorter distances.
- Variance = 11,766.23 and Standard Deviation = 108.47 show that the data is highly spread out.
- 25% of ships travel ≤ 79.0025 units, 50% of ships travel ≤ 123.465 units (median) and 75% of ships travel ≤ 180.78 units of distance.

FOR THE FUEL CONSUMPTION COLUMN:

- As the mean is higher than the median of fuel consumptions of ships, the graph is right skewed.
- Since the mode is much lower than the mean and median, it tells that a large number of ships consume very low amounts of fuel.
- Variance = 23,935,116.04 and Standard Deviation = 4892.35 show that the fuel consumption varies drastically across different ships.
- 25% of ships consume ≤ 1837.96 units, 50% of ships consume ≤ 3060.88 units (median) and 75% of ships consume ≤ 4870.675 units of fuel.

FOR THE CO2 EMISSIONS COLUMN:

- As the mean is higher than the median of CO2 emissions of all ships, the graph is right skewed.
- Since the mode is between the mean and median, it tells that some ships commonly emit around 10625.76 units of CO2.
- Variance = 184081129.73 and Standard Deviation = 13567.65 show the extreme difference in CO2 emissions among ships.
- 25% of ships emit ≤ 4991.49 units, 50% of ships emit ≤ 8423.26 units (median) and 75% of ships emit ≤ 13447.12 units of CO2.

FOR THE ENERGY EFFICIENCY COLUMN:

- As the mean ,mode and median of efficiency of engines are very close , the graph is approximately symmetric.
- This says that engine efficiency is consistent, among all the ships and the distribution is close to normal.
- Variance = 51.24 and Standard Deviation = 7.16 show that there is some variation and values are relatively clustered around the mean.
- 25% of ships have ≤ 76.26 units, 50% of ships have ≤ 82.78 units (median) and 75% of ships have ≤ 88.86 units of energy efficiency

Part-3

Approximating the distribution of sample mean of numerical columns

Plotting density functions

Percentage of data falling within standard intervals has been calculated. Based on the observations, conclusions are made

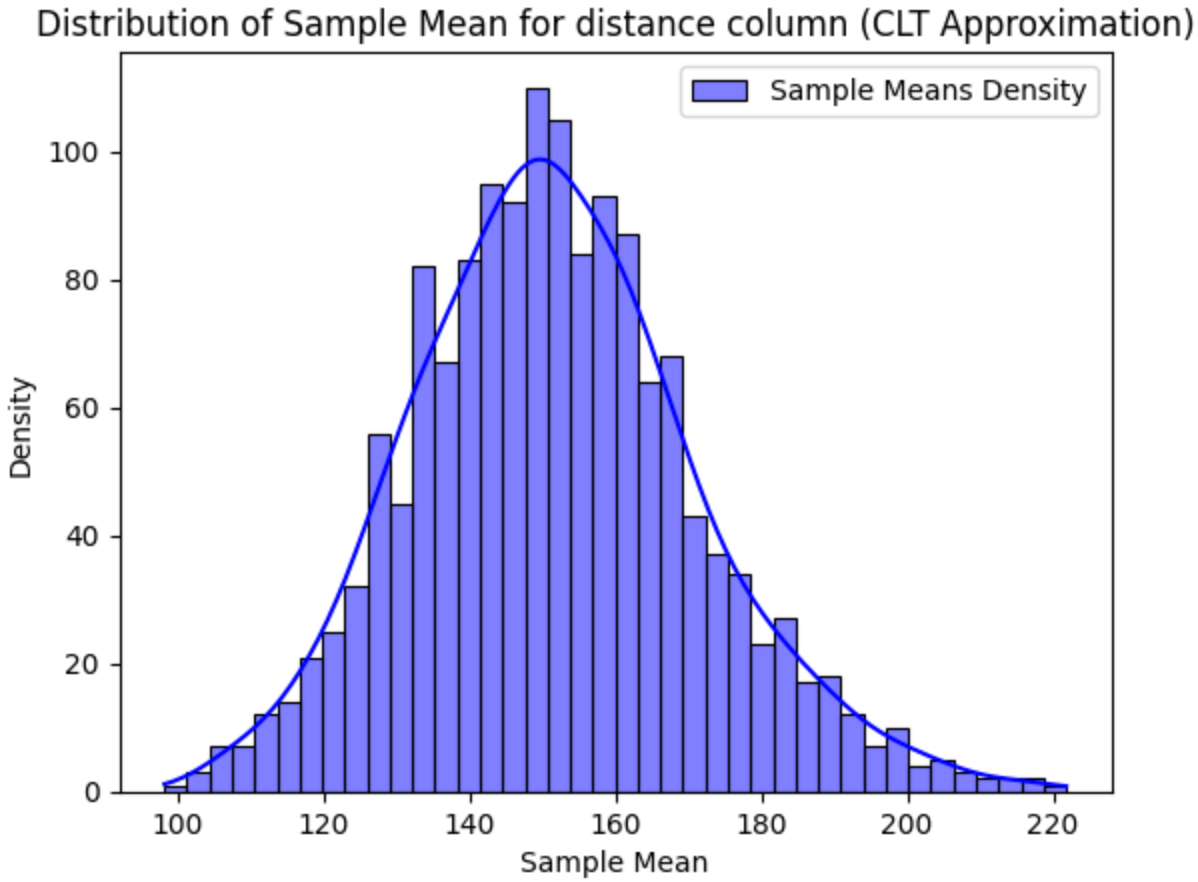
Analyzing the distribution of sample mean using CLT

Code used:

```
import matplotlib.pyplot as plt
import seaborn as sns

sample_size = 30  # Number of data points in each sample
num_samples = 1500
real_data=data['distance']
sample_mean_of_distance = [np.mean(np.random.choice(real_data,
sample_size, replace=True)) for _ in range(num_samples)]


sns.histplot(sample_mean_of_distance, bins=40, kde=True,
color="blue", label="Sample Means Density")
plt.xlabel("Sample Mean")
plt.ylabel("Density")
plt.title("Distribution of Sample Mean for distance column (CLT
Approximation)")
plt.legend()
plt.show()
```

The pdf of distribution of the sample mean looks almost like a gaussian like curve.

We can also get the percentage of data present between the intervels (x-s,x+s) , (x-2.s, x+2.s), (x-3.s,x+3.s) using the code present in the next slide .



```
#mean and std.dev of abv distribution
mean_of_data=np.mean(sample_mean_of_distance)
std_of_data=np.std(sample_mean_of_distance)
print(mean_of_data)
print(std_of_data)

# Percentage of data lying between the standard intervals
lower_bound_1 = mean_of_data - std_of_data
upper_bound_1 = mean_of_data + std_of_data
count_in_range = np.sum((np.array(sample_mean_of_distance) >= lower_bound_1) &
                        (np.array(sample_mean_of_distance) <= upper_bound_1))
percentage = (count_in_range / len(sample_mean_of_distance)) * 100
print(f"Percentage of data between x-s and x+s: {percentage:.2f}%")

lower_bound_2 = mean_of_data - 2 * std_of_data
upper_bound_2 = mean_of_data + 2 * std_of_data
count_in_range = np.sum((np.array(sample_mean_of_distance) >= lower_bound_2) &
                        (np.array(sample_mean_of_distance) <= upper_bound_2))
percentage = (count_in_range / len(sample_mean_of_distance)) * 100
print(f"Percentage of data between x-2s and x+2s: {percentage:.2f}%")

lower_bound_3 = mean_of_data - 3 * std_of_data
upper_bound_3 = mean_of_data + 3 * std_of_data
count_in_range = np.sum((np.array(sample_mean_of_distance) >= lower_bound_3) &
                        (np.array(sample_mean_of_distance) <= upper_bound_3))
percentage = (count_in_range / len(sample_mean_of_distance)) * 100
print(f"Percentage of data between x-3s and x+3s: {percentage:.2f}%")
```

Outputs on running the code:

```
Percentage of data between x-s and x+s: 66.27%
Percentage of data between x-2s and x+2s: 96.13%
Percentage of data between x-3s and x+3s: 99.73%
```

Conclusion:

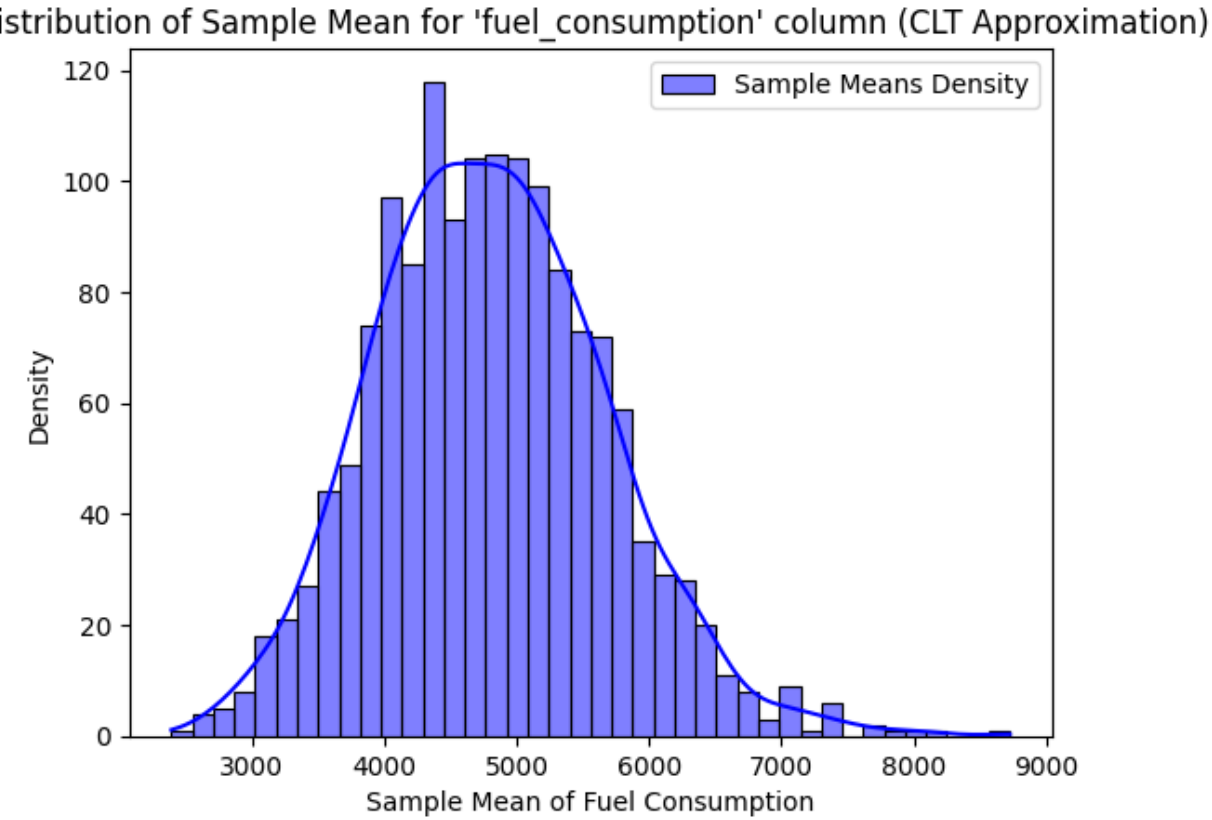
From the above values we can say, the distribution of sample mean of distance column follows a normal distribution approximately

Code used:

```
sample_size = 30  # Number of data points in each sample
num_samples = 1500  # Number of random samples taken

real_data = data['fuel_consumption']
sample_means_of_fuel_consumption =
[np.mean(np.random.choice(real_data, sample_size, replace=True))
for _ in range(num_samples)]

sns.histplot(sample_means_of_fuel_consumption, bins=40, kde=True,
color="blue", label="Sample Means Density")
plt.xlabel("Sample Mean of Fuel Consumption")
plt.ylabel("Density")
plt.title("Distribution of Sample Mean for 'fuel_consumption'
column (CLT Approximation)")
plt.legend()
plt.show()
```

The pdf of distribution of the sample mean of fuel_consumption column looks almost like a gaussian like curve.

We can also get the percentage of data present between the intervels (x-s,x+s) , (x-2.s, x+2.s), (x-3.s,x+3.s) using the code present in the next slide and get a strong observation.



```
#mean and std.dev of abv distribution
mean_of_data=np.mean(sample_means_of_fuel_consumption)
std_of_data=np.std(sample_means_of_fuel_consumption)
print(mean_of_data)
print(std_of_data)

# Percentage of data lying between the standard intervals
lower_bound_1 = mean_of_data - std_of_data
upper_bound_1 = mean_of_data + std_of_data
count_in_range = np.sum((np.array(sample_means_of_fuel_consumption) >= lower_bound_1) &
                        (np.array(sample_means_of_fuel_consumption) <= upper_bound_1))
percentage = (count_in_range / len(sample_means_of_fuel_consumption)) * 100
print(f"Percentage of data between x-s and x+s: {percentage:.2f}%")


lower_bound_2 = mean_of_data - 2 * std_of_data
upper_bound_2 = mean_of_data + 2 * std_of_data
count_in_range = np.sum((np.array(sample_means_of_fuel_consumption) >= lower_bound_2) &
                        (np.array(sample_means_of_fuel_consumption) <= upper_bound_2))
percentage = (count_in_range / len(sample_means_of_fuel_consumption)) * 100
print(f"Percentage of data between x-2s and x+2s: {percentage:.2f}%")


lower_bound_3 = mean_of_data - 3 * std_of_data
upper_bound_3 = mean_of_data + 3 * std_of_data
count_in_range = np.sum((np.array(sample_means_of_fuel_consumption) >= lower_bound_3) &
                        (np.array(sample_means_of_fuel_consumption) <= upper_bound_3))
percentage = (count_in_range / len(sample_means_of_fuel_consumption)) * 100
print(f"Percentage of data between x-3s and x+3s: {percentage:.2f}%")
```

Outputs on running the code:

```
Percentage of data between x-s and x+s:69.27%
Percentage of data between x-2s and x+2s: 95.67%
Percentage of data between x-3s and x+3s: 99.60%
```

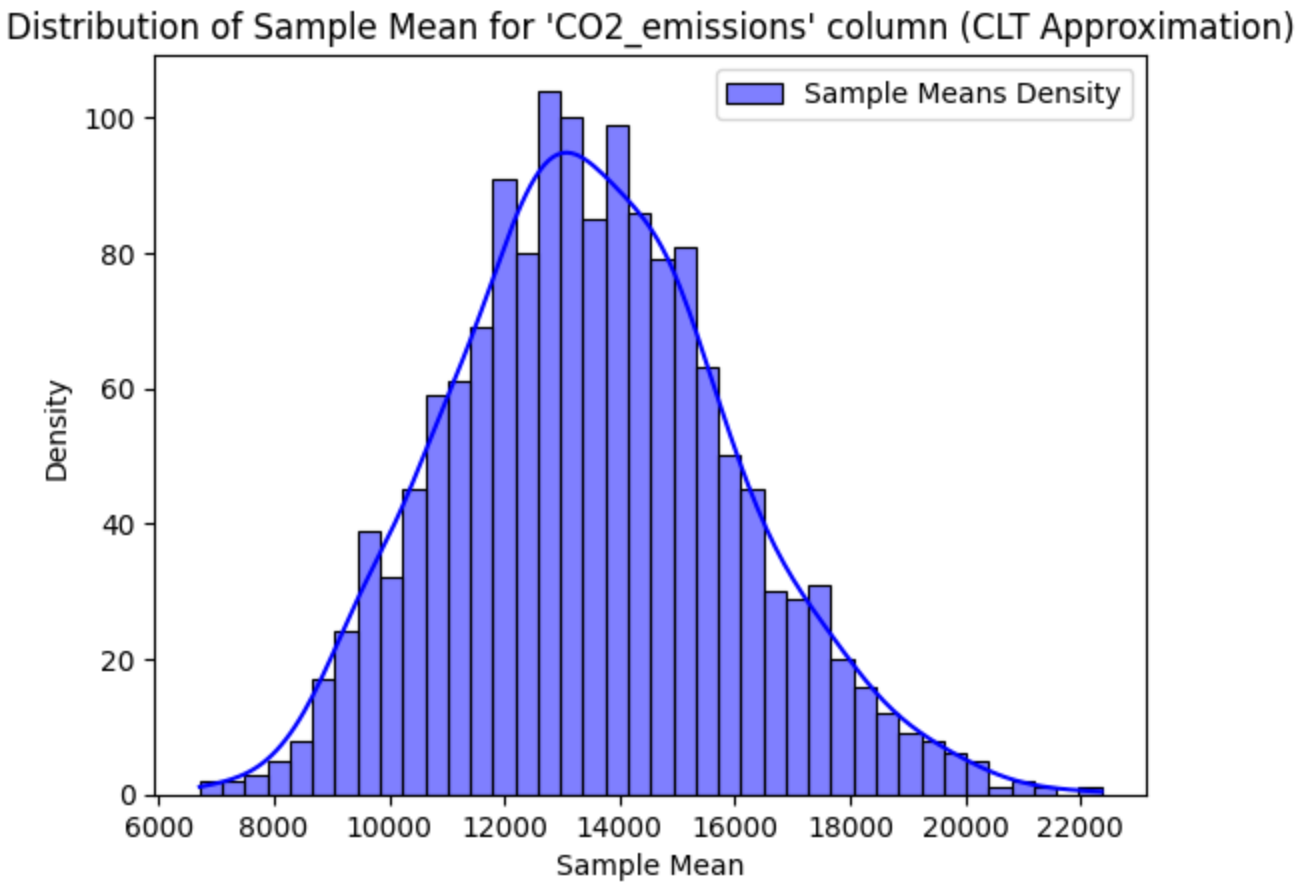
Conclusion:

From the above values we can say, the distribution of sample mean of fuel_consumption column approx. follows a normal distribution

Code used:

```
sample_size = 30  # Number of data points in each sample
num_samples = 1500
real_data=data['CO2_emissions']
sample_means_of_co2_emissions =
[np.mean(np.random.choice(real_data, sample_size,
replace=True)) for _ in range(num_samples)]
```

```
sns.histplot(sample_means_of_co2_emissions, bins=40, kde=True,
color="blue", label="Sample Means Density")
plt.xlabel("Sample Mean")
plt.ylabel("Density")
plt.title("Distribution of Sample Mean for 'CO2_emssions'
column (CLT Approximation)")
plt.legend()
plt.show()
```



The pdf of distribution of the sample mean of fuel_consumption column looks almost like a gaussian like curve.

We can also get the percentage of data present between the intervels (x-s,x+s) , (x-2.s, x+2.s), (x-3.s,x+3.s) using the code present in the next slide and get a strong observation.

```
mean_of_data = np.mean(sample_means_of_co2_emissions)
std_of_data = np.std(sample_means_of_co2_emissions)

lower_bound_1 = mean_of_data - std_of_data
upper_bound_1 = mean_of_data + std_of_data
count_in_range = np.sum((np.array(sample_means_of_co2_emissions) >= lower_bound_1) &
                        (np.array(sample_means_of_co2_emissions) <= upper_bound_1))
percentage = (count_in_range / len(sample_means_of_co2_emissions)) * 100
print(f"Percentage of data between x-s and x+s: {percentage:.2f}%")

lower_bound_2 = mean_of_data - 2 * std_of_data
upper_bound_2 = mean_of_data + 2 * std_of_data
count_in_range = np.sum((np.array(sample_means_of_co2_emissions) >= lower_bound_2) &
                        (np.array(sample_means_of_co2_emissions) <= upper_bound_2))
percentage = (count_in_range / len(sample_means_of_co2_emissions)) * 100
print(f"Percentage of data between x-2s and x+2s: {percentage:.2f}%")

lower_bound_3 = mean_of_data - 3 * std_of_data
upper_bound_3 = mean_of_data + 3 * std_of_data
count_in_range = np.sum((np.array(sample_means_of_co2_emissions) >= lower_bound_3) &
                        (np.array(sample_means_of_co2_emissions) <= upper_bound_3))
percentage = (count_in_range / len(sample_means_of_co2_emissions)) * 100
print(f"Percentage of data between x-3s and x+3s: {percentage:.2f}%")
```

Outputs on running the code:

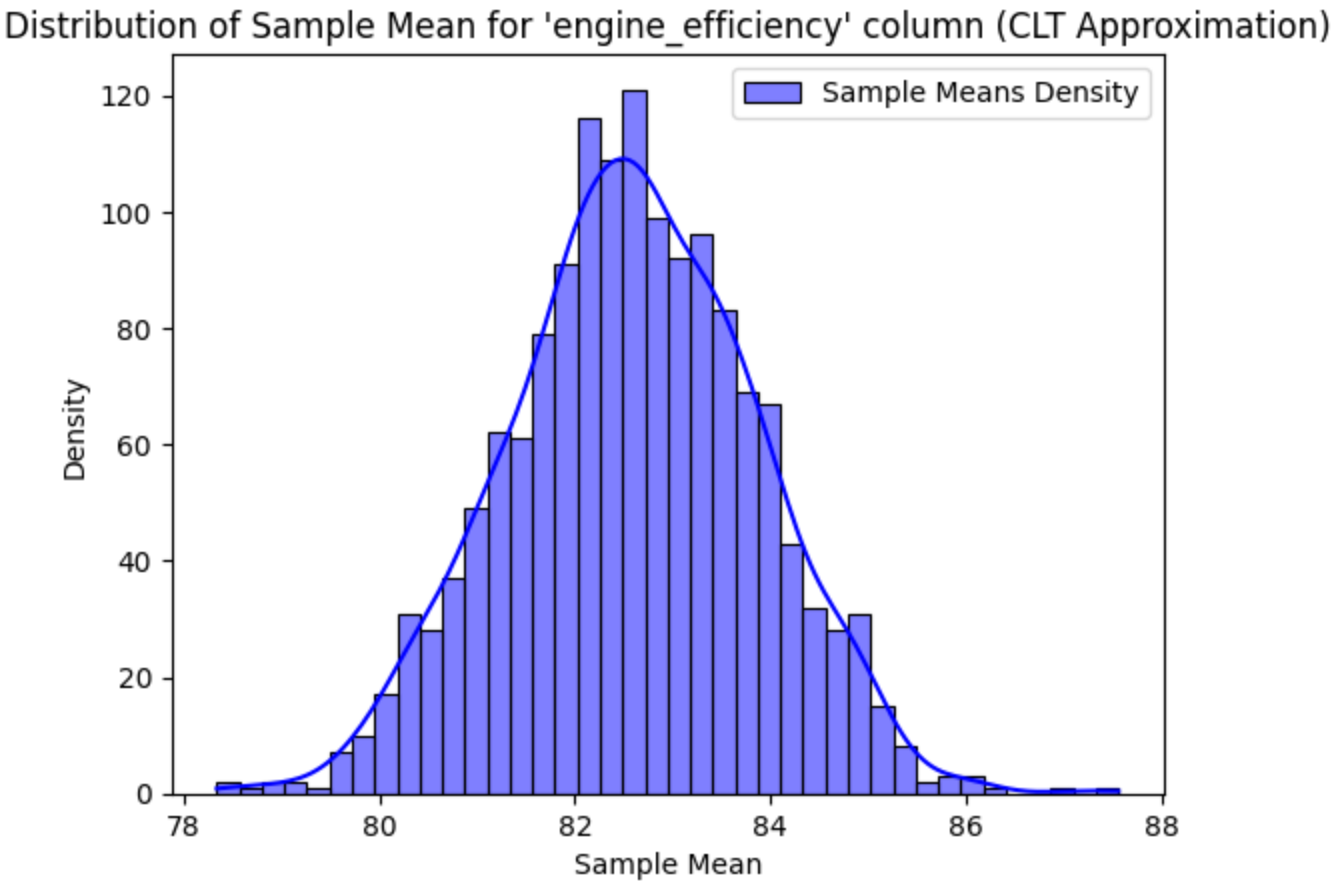
```
Percentage of data between x-s and x+s:69.13%
Percentage of data between x-2s and x+2s: 95.73%
Percentage of data between x-3s and x+3s: 99.47%
```

Conclusion:

From the above values we can say, the distribution of sample mean of CO2_emssions column approx. follows a normal distribution

Code used:

```
import matplotlib.pyplot as plt
import seaborn as sns
sample_size = 30  # Number of data points in each sample
num_samples = 1500
real_data=data['engine_efficiency']
sample_means_of_engine_eff =
[np.mean(np.random.choice(real_data, sample_size,
replace=True)) for _ in range(num_samples)]

sns.histplot(sample_means_of_engine_eff, bins=40, kde=True,
color="blue", label="Sample Means Density")
plt.xlabel("Sample Mean")
plt.ylabel("Density")
plt.title("Distribution of Sample Mean for
'engine_efficiency' column (CLT Approximation)")
plt.legend()
plt.show()
```



The pdf of distribution of the sample mean of engine_efficiency column looks almost like a gaussian like curve.

We can also get the percentage of data present between the intervels (x-s,x+s) , (x-2.s, x+2.s), (x-3.s,x+3.s) using the code present in the next slide and get a strong observation.

```
mean_of_data = np.mean(sample_means_of_engine_eff)
std_of_data = np.std(sample_means_of_engine_eff)

# Percentage of data lying between the standard intervals
lower_bound_1 = mean_of_data - std_of_data
upper_bound_1 = mean_of_data + std_of_data
count_in_range = np.sum((np.array(sample_means_of_engine_eff) >= lower_bound_1) &
                        (np.array(sample_means_of_engine_eff) <= upper_bound_1))
percentage = (count_in_range / len(sample_means_of_engine_eff)) * 100
print(f"Percentage of data between x-s and x+s: {percentage:.2f}%")

lower_bound_2 = mean_of_data - 2 * std_of_data
upper_bound_2 = mean_of_data + 2 * std_of_data
count_in_range = np.sum((np.array(sample_means_of_engine_eff) >= lower_bound_2) &
                        (np.array(sample_means_of_engine_eff) <= upper_bound_2))
percentage = (count_in_range / len(sample_means_of_engine_eff)) * 100
print(f"Percentage of data between x-2s and x+2s: {percentage:.2f}%")

lower_bound_3 = mean_of_data - 3 * std_of_data
upper_bound_3 = mean_of_data + 3 * std_of_data
count_in_range = np.sum((np.array(sample_means_of_engine_eff) >= lower_bound_3) &
                        (np.array(sample_means_of_engine_eff) <= upper_bound_3))
percentage = (count_in_range / len(sample_means_of_engine_eff)) * 100
print(f"Percentage of data between x-3s and x+3s: {percentage:.2f}%")
```

Outputs on running the code:

```
Percentage of data between x-s and x+s:68.13%
Percentage of data between x-2s and x+2s: 95.40%
Percentage of data between x-3s and x+3s: 99.67%
```

Conclusion:

From the above values we can say, the distribution of sample mean of engine-efficiency column approx. follows a normal distribution

We can conclude that the distribution of sample means of all numerical columns follows approximately normal distribution
As discussed already , the CLT approximation works well if we have sufficient no.of samples
We are taking 1500 samples of sample_means which is a good number. So it fits good as a gaussian

The percentage of data present btwn the standard intervals almost equal to that of gaussian distribution
So , we can say they follow an approximately normal distribution

------Thankyou-------

Done by:
- Rishitha(ai23btech11020)
- Harini(ai23btech11034)
- Rama Harshini(ai23btech11028)